DIGITAL DOCUMENT VERIFICATION SYSTEM (DDVS)

Software Metrics and Complexity Analysis Report

Course: B.Tech Computer Science and Engineering	**Experiment Number:** Practical Lab 10
Subject: Software Engineering Laboratory	**Academic Year:** 2026 – 2027
Author / Student: Divya Kumari (202401100200133)	**Institution:** KIET Group of Institutions
Document Version: 1.0 (Final Academic Baseline)	**Date of Evaluation:** November 2026

Executive Summary

This quantitative engineering report performs a formal Software Metrics and Complexity Analysis of the Digital Document Verification System (DDVS). Grounded in established Software Engineering measurement theory, the report evaluates product, process, and project metrics across the architectural models formulated in Labs 01 through 09. McCabe's Cyclomatic Complexity is derived from a formal Control Flow Graph (CFG) of the core verification engine, proving a complexity rating of V(G) = 5 with exactly five independent basis paths. An honest evaluation of Lines of Code (LOC) clarifies why physical implementation code cannot yet be measured at the design stage, providing design-based estimations instead. Chidamber & Kemerer (CK) Object-Oriented Metrics—including WMC, DIT, NOC, CBO, RFC, and LCOM—are rigorously calculated from the UML Class Diagram. The results demonstrate that DDVS exhibits low algorithmic risk, high testability, and balanced structural complexity.

1. Introduction to Software Measurement and Metrics

In Software Engineering, measurement provides quantitative insight into software processes, project health, and product quality. Without metrics, software quality remains subjective and untestable. Metrics are categorized into three core domains:

- **Product Metrics:** Quantify intrinsic characteristics of software artifacts (specifications, architecture, designs, source code). Examples: Cyclomatic Complexity, Lines of Code, Coupling Between Objects (CBO), Depth of Inheritance (DIT).
- **Process Metrics:** Quantify the efficiency and effectiveness of the software engineering process. Examples: Defect arrival rate, bug removal efficiency, lead time for feature delivery.
- **Project Metrics:** Quantify resource allocation and operational progress. Examples: Staff effort (person-months), budget burn rate, schedule variance, milestone completion rates.

2. Selected Metric Suite Definitions and Theoretical Formulas

Metric Name	Theoretical Formula	Target Attribute	Acceptable Industry Threshold
Cyclomatic Complexity (McCabe)	V(G) = E - N + 2P = Pdec + 1	Algorithmic complexity, testability	1 – 10: Simple, low risk 11 – 20: Moderate risk > 20: High risk
Weighted Methods / Class (WMC)	WMC = ∑ c_i (sum of method complexities)	Class complexity, maintenance effort	1 – 20 per class
Depth of Inheritance Tree (DIT)	DIT = Max path length from root to node	Inheritance depth, design reusability	0 – 3 (shallow trees preferred)
Number of Children (NOC)	NOC = Count of immediate subclasses	Subclass reuse, testing impact	0 – 4 per superclass
Coupling Between Objects (CBO)	CBO = Count of external coupled classes	Modularity, independence	0 – 5 (lower is better)
Response For a Class (RFC)	RFC = |M| + ∑ |R_i| (methods + called)	Class communication complexity	< 50 per class
Lack of Cohesion in Methods (LCOM)	LCOM = |P| - |Q| if |P| > |Q| else 0	Single responsibility, class cohesion	0 (perfect cohesion) to Low

3. McCabe Cyclomatic Complexity Analysis of DDVS Verification Engine

Thomas McCabe's Cyclomatic Complexity metric measures the number of linearly independent paths through a program's control flow graph (CFG). It establishes the minimum number of test cases required to achieve 100% basis path coverage.

3.1 Graph Theoretic Derivation

From the formal Control Flow Graph (CFG) of the DDVS Verification Algorithm, the mathematical parameters are: • Total Nodes (N) = 14 (representing discrete computational steps and decision forks) • Total Edges (E) = 17 (representing transfers of control) • Connected Components (P) = 1 (a single, unified procedure) • Predicate Decision Nodes (Pdec) = 4 (Nodes 3, 6, 8, 11)

Formula 1 (Graph Topology):
V(G) = E - N + 2P = 17 - 14 + 2(1) = 3 + 2 = **5**

Formula 2 (Predicate Node Formulation):
V(G) = Pdec + 1 = 4 + 1 = **5**

Formula 3 (Planar Graph Enclosed Regions):
V(G) = R = 4 (enclosed bounded planar faces) + 1 (unbounded external region) = **5**

3.2 Identification of the Five Independent Basis Paths

- **Basis Path 1 (Record Not Found):** Node 1 ■■■ 2 ■■■ 3 ■■■ 4 ■■■ 14 (Exit)

- **Basis Path 2 (Tampered Credential):** Node 1 ■■■ 2 ■■■ 3 ■■■ 5 ■■■ 6 ■■■ 7 ■■■ 8 ■■■ 9 ■■■ 14 (Exit)
- **Basis Path 3 (Revoked / Invalid with File):** Node 1 ■■■ 2 ■■■ 3 ■■■ 5 ■■■ 6 ■■■ 7 ■■■ 8 ■■■ 10 ■■■ 11 ■■■ 12 ■■■ 14 (Exit)
- **Basis Path 4 (Genuine Verified with File):** Node 1 ■■■ 2 ■■■ 3 ■■■ 5 ■■■ 6 ■■■ 7 ■■■ 8 ■■■ 10 ■■■ 11 ■■■ 13 ■■■ 14 (Exit)
- **Basis Path 5 (Genuine Verified via ID/QR Only):** Node 1 ■■■ 2 ■■■ 3 ■■■ 5 ■■■ 6 ■■■ 10 ■■■ 11 ■■■ 13 ■■■ 14 (Exit)

Complexity Interpretation: A Cyclomatic Complexity of **V(G) = 5** falls squarely in the 'Low Complexity, High Testability' band (1 – 10). This indicates that the core verification algorithm is simple, robust, easily comprehensible, and requires only five unit test fixtures to achieve 100% path coverage.

4. Lines of Code (LOC) Analysis: Architectural Stage Assessment

4.1 Academic Measurement Integrity and Counting Conventions

Lines of Code (LOC) can be counted as Physical LOC (raw text lines) or Source Lines of Code (SLOC / Logical LOC, counting executable statements excluding whitespace and comments). In software engineering rigor, **LOC can only be measured after physical code is authored**.

Because the DDVS project is currently at the **Software Design and Architectural Modeling stage (Labs 01–09)**, no backend implementation code has been written yet. It is academically dishonest to fabricate measured LOC numbers. Instead, we provide **Design Estimates** based on standard object-oriented implementation density (assuming ~25–35 SLOC per method):

Class / Component	UML Method Count	Estimated SLOC / Method	Estimated Class SLOC	Metric Classification
User	5	20 – 30	100 – 150	Design Estimate
IssuingOrganization	4	25 – 35	100 – 140	Design Estimate
Administrator	5	20 – 30	100 – 150	Design Estimate
Document	4	25 – 35	100 – 140	Design Estimate
VerificationRequest	3	20 – 30	60 – 90	Design Estimate
VerificationLog	3	20 – 30	60 – 90	Design Estimate
Total Core Domain	**24**	**~25**	**520 – 760 SLOC**	**Architectural Estimate**

5. Object-Oriented Design Metrics Evaluation (Chidamber & Kemerer Suite)

Using the structural UML Class Diagram established in Lab 03, the six Chidamber & Kemerer (CK) metrics were calculated for all identified domain classes:

Class Name	WMC	DIT	NOC	CBO	RFC	LCOM	Metric Type

User	5	0	2	2	5	0 (High)	Design Calculated
IssuingOrganization	4	1	0	2	9	0 (High)	Design Calculated
Administrator	5	1	0	2	10	0 (High)	Design Calculated
Document	4	0	0	3	4	0 (High)	Design Calculated
VerificationRequest	3	0	0	3	3	0 (High)	Design Calculated
VerificationLog	3	0	0	3	3	0 (High)	Design Calculated

6. Software Metrics Synthesis and Status Classification

Metric Name	Formula / Definition	DDVS Design Value	Interpretation & Quality Evaluation	Classification
Cyclomatic Complexity V(G)	E - N + 2P	5	Low complexity, high testability (5 basis paths)	Calculated (CFG)
Source Lines of Code (SLOC)	Executable statements	520 – 760 (est.)	Compact domain logic; actual code pending	Design Estimate
Number of Classes (NC)	Count of domain classes	6	Modular, cohesive domain architecture	Calculated (UML)
Total Operations (NOM)	Sum of public methods	24	Well-distributed method responsibilities	Calculated (UML)
Max Inheritance Depth (DIT)	Max tree depth	1	Shallow hierarchy avoids fragile base class	Calculated (UML)
Mean Coupling (CBO)	Avg external couplings	2.5	Low coupling promotes independent testability	Calculated (UML)

7. Architectural Recommendations for Complexity Management

To maintain low complexity during subsequent physical implementation, the following architectural guidelines are recommended: 1. **Preserve Small, Single-Task Methods:** Maintain method lengths under 30 lines of code to prevent cyclomatic inflation. 2. **Avoid Deep Inheritance Hierarchies:** Keep DIT ≤ 2. If additional user privileges or issuer types are required, prefer composition and strategy patterns over deep inheritance. 3. **Enforce Interface Segregation:** Keep CBO low by depending exclusively on narrow interfaces (`IHashService`, `IDataAccess`) rather than concrete classes. 4. **Automate Static Metric Audits:** Integrate linter and complexity analyzers (e.g., SonarQube, radon, ESLint) into the Continuous Integration (CI) pipeline to reject pull requests exceeding V(G) > 10.

8. Conclusion and Future Quantitative Measurement Plan

The complexity analysis confirms that the Digital Document Verification System is characterized by sound mathematical and architectural design. McCabe's Cyclomatic Complexity V(G) = 5 confirms that verification logic is concise, predictable, and completely covered by five test paths. The CK metrics confirm low inheritance depth (DIT = 1) and minimal inter-class coupling (mean CBO = 2.5). While actual physical LOC and defect removal metrics will be captured during post-coding test phases, the architectural baseline guarantees high maintainability and low defect probability.